LABORATORY WORK NO. 4

# Block Ciphers. The DES Algorithm

*Task 2.5 — given $K_+$, determine all sixteen round keys*

---


Discipline: **Cryptography**

Author: **Mutu Adrian**

Group: **FAF-242**


1 November 2026

# 1. Short theory

The ciphers of the first three laboratory works operate on one letter at a time. A *block cipher* works instead on fixed-size groups of bits, transforming a whole block under the control of a key, and it is the form in which practically all modern symmetric encryption is done. The Data Encryption Standard, published as FIPS 46 in 1977 and withdrawn in 2005, is the block cipher every later design is measured against: it encrypts **64-bit blocks** under a **64-bit key of which only 56 bits are used**, in **16 rounds** of a Feistel network.

## 1.1 The structure of the algorithm

A DES encryption of one block runs as follows:

1. the 64 bits of the block are rearranged by the initial permutation `IP` and split into two halves of 32 bits, $L_0$ and $R_0$;
2. sixteen rounds are applied. Round `i` keeps the right half as the new left half and adds the output of the round function to the old left half:

$$L_i = R_{i-1} \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$$

3. after round 16 the two halves are swapped once more and the 64 bits $R_{16}L_{16}$ are rearranged by $IP^{-1}$, the inverse of the initial permutation, giving the ciphertext.

This shape is called a *Feistel network*, and it has one property that matters above all others: the round function `f` need not be invertible. Whatever `f` does, the round can be undone by knowing $R_{i-1} = L_i$ and computing $L_{i-1} = R_i \oplus f(L_i, K_i)$. Decryption is therefore the very same algorithm with the sixteen round keys applied in the reverse order, $K_{16}$ first — which is why the key schedule is worth implementing on its own, as this task does.

## 1.2 The round function

The function `f(R, K)` turns 32 bits and a 48-bit round key into 32 bits in four steps:

- **Expansion.** `E` stretches the 32 bits of `R` to 48 by repeating 16 of them, so that the block and the round key have the same length.
- **Key mixing.** The round key is added bit by bit modulo 2: `E(R) ⊕ K`. This is the only place the key enters the data path, and it is why the sixteen round keys are the whole of what the key contributes to the cipher.
- **Substitution.** The 48 bits are cut into eight groups of six, $B_1 \dots B_8$, and each group enters its own S-box, which replaces 6 bits with 4: the outer bits $b_1b_6$ choose one of four rows, the inner bits $b_2b_3b_4b_5$ one of sixteen columns. The eight S-boxes are the only non-linear part of DES and the reason it resists the attacks of the previous laboratory works.
- **Permutation.** The 32 bits that come out are rearranged by `P`, which spreads the output of each S-box across the inputs of several S-boxes of the next round — the diffusion of the cipher.

## 1.3 The key schedule — the subject of this work

The sixteen round keys are produced from the key alone, before any data is touched:

$$K \ \text{—PC-1→}\ K^{+} \ \rightarrow \ C_0D_0$$

$$C_i = \mathrm{LS}_i(C_{i-1}) \quad D_i = \mathrm{LS}_i(D_{i-1}) \quad K_i = \text{PC-2}(C_iD_i)$$

**PC-1** (Permuted Choice 1) selects 56 of the 64 bits of the key and reorders them, giving `K+`. The eight bits it leaves out are those in positions 8, 16, 24, …, 64 — the parity bit of each byte, which carries no key material. This is the reason a "64-bit" DES key has an effective length of only 56 bits.

`K+` is split into two halves of 28 bits, `C₀` and `D₀`. In round `i` each half is rotated to the left on its own by `LSᵢ` places — one place in rounds 1, 2, 9 and 16, and two in the other twelve. The sixteen shifts add up to 28, the length of a half, so after the last round the halves are back where they started: `C₁₆ = C₀` and `D₁₆ = D₀`. That is a useful check on any implementation, and it is also what lets the same schedule be generated in reverse for decryption.

**PC-2** (Permuted Choice 2) then selects 48 of the 56 bits of `CᵢDᵢ` and reorders them, giving the round key `Kᵢ`. Eight positions — 9, 18, 22, 25, 35, 38, 43 and 54 — are never named by PC-2 and are therefore dropped in every round; because the halves rotate, a bit dropped in one round is used in the next, so every key bit takes part in about fourteen of the sixteen rounds.

### 1.4 Security

DES was never broken mathematically in the way the ciphers of laboratory works 1 to 3 were: after more than forty years of analysis, differential and linear cryptanalysis are the best attacks known and both need quantities of chosen or known plaintext that are impractical. What defeated DES was the key length. A 56-bit key allows only `2⁵⁶ ≈ 7.2 × 10¹⁶` possibilities, and exhaustive search became feasible in hardware: the EFF "Deep Crack" machine recovered a key in 56 hours in 1998. The response was Triple DES, which applies the algorithm three times with two or three keys, and then AES, which replaced DES as the standard in 2001.

## 2. The assigned task

The assignment selects the task from the list by `nr_sarcina = n mod 11`, where `n` is the student's number in the group register. With `n = 16`:

$$\text{nr_sarcina} = 16 \bmod 11 = 5$$

which is task 2.5: "*În algoritmul DES este dat K+. Să se determine toate cele 16 chei de rundă $K_i$.*" — given `K+`, determine all sixteen round keys. The assignment further requires that the tables used and all the intermediate steps be shown on screen, and that the input data can either be typed by the user or generated at random.

## 3. Implementation

The program is written in Python 3 and uses only the standard library.

| File | Contents |
|---|---|
| `bits.py` | bit strings and the operations DES performs on them |
| `des_tables.py` | PC-1, the rotation schedule $LS_i$ and PC-2 |
| `key_schedule.py` | K+ → CoDo → the sixteen round keys, with every intermediate value |
| `main.py` | the interactive program |
| `tests/test_key_schedule.py` | the standard worked example and the structural properties |
| `tests/des_reference.py`, `tests/test_known_answer.py` | the DES data path, used only to verify the schedule |

### Bit strings

A block is a plain list of 0/1 integers. FIPS 46-3 numbers the bits of a block from 1 on the left and writes every table in those terms, so the permutation function indexes its table 1-based and nothing in the program has to re-number anything:

```
def permute(block, table):
    """Return the block rearranged by `table`, a 1-based list of positions."""
    return [block[position - 1] for position in table]


def rotate_left(block, amount):
    """Rotate a block one or more places to the left (the LS_i of the standard)."""
    amount %= len(block)
    return block[amount:] + block[:amount]
```

One function therefore performs both PC-1 and PC-2 — and, in the data path, IP, E, P and $IP^{-1}$ as well, since in DES every table is read the same way: *the bit of the output at position i is the bit of the input at position table[i]*. That single observation is what keeps the implementation short.

### The tables

PC-1, PC-2 and the shift schedule are transcribed from the standard. Two facts about them are computed rather than written down, so that a typing mistake in a table would be caught instead of being believed:

```
PARITY_BITS = tuple(position for position in range(1, KEY_BITS + 1)
                    if position not in PC1)

DROPPED_BY_PC2 = tuple(position for position in range(1, KPLUS_BITS + 1)
                       if position not in PC2)
```

These produce `(8, 16, 24, 32, 40, 48, 56, 64)` and `(9, 18, 22, 25, 35, 38, 43, 54)`, which are exactly the positions the standard says are discarded — a first confirmation that the tables were copied correctly.

### The key schedule

```
def permuted_choice_1(key):
    """K+ = PC-1(K): drop the 8 parity bits of the key and reorder the rest."""
    return bits.permute(key, PC1)


def split(kplus):
    """C_0 and D_0: the left and the right half of K+."""
    return kplus[:HALF_BITS], kplus[HALF_BITS:]


def halves(kplus):
    """All the pairs (C_i, D_i), from i = 0 to i = 16."""
    c, d = split(kplus)
    pairs = [(c, d)]
    for shift in SHIFTS:
        c = bits.rotate_left(c, shift)
        d = bits.rotate_left(d, shift)
        pairs.append((c, d))
    return pairs


def round_key(c, d):
    """K_i = PC-2(C_i D_i): 48 of the 56 bits of the two halves."""
    return bits.permute(c + d, PC2)


def round_keys(kplus):
    """The sixteen round keys K_1 ... K_16 - the answer to task 2.5."""
    return [round_key(c, d) for c, d in halves(kplus)[1:]]
```

A further function, `schedule()`, returns one record per round holding the shift applied, both halves before and after it and the resulting round key. The cipher itself needs only `round_keys()`; `schedule()` exists because the assignment requires every intermediate step to be displayed, and keeping the two apart means the printing code never has to recompute anything.

### Input

The program accepts the key as **8 characters** — the "8 simboluri" of the assignment, one byte each — or as **16 hexadecimal digits**, which is how published DES test keys are written. Since 16 hexadecimal digits are never 8 characters, the two cannot be confused. `K+` may be typed directly as 56 bits or 14 hexadecimal digits, and a key can be generated at random. Everything that is rejected is reported with the reason, and the question is asked again.

## 4. Results

The runs below were produced by `python3 main.py` and are reproduced as they appear on screen. The key `133457799BBCDFF1` is used throughout, because it is the key of the standard worked example of DES and every value can therefore be compared against a published one.

### The tables

```
================================================================================
The tables of the DES key schedule
================================================================================
  PC-1 - chooses 56 of the 64 bits of K:
    57 49 41 33 25 17  9
     1 58 50 42 34 26 18
    10  2 59 51 43 35 27
    19 11  3 60 52 44 36
    63 55 47 39 31 23 15
     7 62 54 46 38 30 22
    14  6 61 53 45 37 29
    21 13  5 28 20 12  4

    dropped (parity bits): 8, 16, 24, 32, 40, 48, 56, 64

  LS_i - the left rotation of round i:
    round   1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16
    shift   1  1  2  2  2  2  2  2  1  2  2  2  2  2  2  1

  PC-2 - chooses 48 of the 56 bits of C_i D_i:
    14 17 11 24  1  5
     3 28 15  6 21 10
    23 19 12  4 26  8
    16  7 27 20 13  2
    41 52 31 37 47 55
    30 40 51 45 33 48
    44 49 39 56 34 53
    46 42 50 36 29 32

    dropped: 9, 18, 22, 25, 35, 38, 43, 54
```

### From the key to K+

```
Your choice: 1
Key - 8 characters or 16 hexadecimal digits: 133457799BBCDFF1

========================================================================
Task 2.5 - the sixteen round keys of DES, starting from the key
========================================================================
  The key K (64 bits):
    hex       133457799BBCDFF1
    bits      00010011 00110100 01010111 01111001
              10011011 10111100 11011111 11110001

------------------------------------------------------------------------
Step 1 - PC-1: K  ->  K+   (56 of the 64 bits, reordered)
------------------------------------------------------------------------
  The 8 bits PC-1 leaves out are the parity bits of the key -
  positions 8, 16, 24, 32, 40, 48, 56, 64 - which carry no key material.
    parity bits of K: 1 0 1 1 1 0 1 1

  K+ (56 bits, in groups of 7):
    1111000 0110011 0010101 0101111 0101010 1011001 1001111 0001111
    hex F0CCAAF556678F

------------------------------------------------------------------------
Step 2 - split K+ into two halves of 28 bits
------------------------------------------------------------------------
  C0 = 1111000011001100101010101111
  D0 = 0101010101100110011110001111

------------------------------------------------------------------------
Step 3 - the rotation schedule LS_i
------------------------------------------------------------------------
    round   1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16
    shift   1  1  2  2  2  2  2  2  1  2  2  2  2  2  2  1
  The sixteen shifts add up to 28, the length of a half, so after
  round 16 the halves are back where they started: C16 = C0.
```

The PC-1 table itself is printed at this point too; it is omitted here because it appears in the previous listing.

### The rounds

Each of the sixteen rounds prints the shift, both halves before and after it, and the round key. The first two and the last are reproduced here; the remaining thirteen have the same form.

```
Round 1   (rotate both halves left by 1)
  C0  = 11110000110011001010101111
  D0  = 01010101011001100111100011111
  C1  = 11100001100110010101011111
  D1  = 10101010110011001111100011110
  K1  = 000110 110000 001011 101111 111111 000111 000001 110010
        hex 1B02EFFC7072

Round 2   (rotate both halves left by 1)
  C1  = 11100001100110010101011111
  D1  = 10101010110011001111100011110
  C2  = 11000011001100101010111111
  D2  = 01010101100110011111000111101
  K2  = 011110 011010 111011 011001 110110 111100 100111 100101
        hex 79AED9DBC9E5

...

Round 16   (rotate both halves left by 1)
  C15 = 11111000011001100101010111
  D15 = 10101010101100110011111000111
  C16 = 11110000110011001010101111
  D16 = 01010101011001100111100011111
  K16 = 110010 110011 110110 001011 000011 100001 011111 110101
        hex CB3D8B0E17F5
```

In round 16 the two halves printed as `C16` and `D16` are literally the `C0` and `D0` of step 2 — the shifts have closed the circle, exactly as section 1.3 predicts.

### The answer — the sixteen round keys

```
------------------------------------------------------------------------
The sixteen round keys
------------------------------------------------------------------------
  K1  = 000110 110000 001011 101111 111111 000111 000001 110010   (1B02EFFC7072)
  K2  = 011110 011010 111011 011001 110110 111100 100111 100101   (79AED9DBC9E5)
  K3  = 010101 011111 110010 001010 010000 101100 111110 011001   (55FC8A42CF99)
  K4  = 011100 101010 110111 010110 110110 110011 010100 011101   (72ADD6DB351D)
  K5  = 011111 001110 110000 000111 111010 110101 001110 101000   (7CEC07EB53A8)
  K6  = 011000 111010 010100 111110 010100 000111 101100 101111   (63A53E507B2F)
  K7  = 111011 001000 010010 110111 111101 100001 100010 111100   (EC84B7F618BC)
  K8  = 111101 111000 101000 111010 110000 010011 101111 111011   (F78A3AC13BFB)
  K9  = 111000 001101 101111 101011 111011 011110 011110 000001   (E0DBEBEDE781)
  K10 = 101100 011111 001101 000111 101110 100100 011001 001111   (B1F347BA464F)
  K11 = 001000 010101 111111 010011 110111 101101 001110 000110   (215FD3DED386)
  K12 = 011101 010111 000111 110101 100101 000110 011111 101001   (7571F59467E9)
  K13 = 100101 111100 010111 010001 111110 101011 101001 000001   (97C5D1FABA41)
  K14 = 010111 110100 001110 110111 111100 101110 011100 111010   (5F43B7F2E73A)
  K15 = 101111 111001 000110 001101 001111 010011 111100 001010   (BF918D3D3F0A)
  K16 = 110010 110011 110110 001011 000011 100001 011111 110101   (CB3D8B0E17F5)

  Check: C16 = C0 and D16 = D0 ... yes
```

### Task 2.5 as it is stated — K+ given

The task gives `K+`, not the key. The second menu entry takes it directly, and the key itself is never needed:

```
Your choice: 2
K+ - 56 bits (0s and 1s) or 14 hexadecimal digits:
11110000110011001010101011110101010101100110011110001111

========================================================================
Task 2.5 - the sixteen round keys of DES, starting from K+
========================================================================
  K+ (56 bits, in groups of 7):
    1111000 0110011 0010101 0101111 0101010 1011001 1001111 0001111
    hex F0CCAAF556678F
```

From there the output is identical to the listing above, which is the point: `K+` is all the key schedule ever uses.

### An 8-character key

Typing the key as eight characters prints their ASCII codes as well, since that is where the 64 bits come from:

```
Key - 8 characters or 16 hexadecimal digits: Cripto24

  The key K (64 bits):
    character   C    r    i    p    t    o    2    4
    ASCII      67  114  105  112  116  111   50   52
    hex       43726970746F3234
    bits      01000011 01110010 01101001 01110000
              01110100 01101111 00110010 00110100

  K+ (56 bits, in groups of 7):
    0000000 0001111 1111111 1101101 0110001 1101100 0000100 1001010
    hex 003FFED63B024A

  C0 = 00000000001111111111111101101
  D0 = 01100011101100000001001001010
  K1  = 111100 001011 011000 101110 010000 100000 011010 100110   (F0B62E4206A6)
  K16 = 111000 011010 111010 100110 001111 110000 001001 001100   (E1AEA63F024C)
```

The long run of zeros at the start of this `K+` is not a fault: printable ASCII characters all have bit 1 equal to 0, and PC-1 gathers the first column of the key into the first bits of `K+`. It is a reminder that a key made of typed characters is far weaker than its 56 bits suggest — a point the third menu entry avoids by generating all 64 bits at random.

### Invalid input

```
Your choice: 5
  [invalid input] '5' is not one of the options; type 0 or 1 or 2 or 3 or 4

Your choice: 1
Key - 8 characters or 16 hexadecimal digits: Criptoăă
  [invalid input] the character 'ă' has no single-byte code, so it cannot be
  one of the 8 symbols of a DES key; use characters with an ASCII code, or
  give the key as 16 hexadecimal digits

Key - 8 characters or 16 hexadecimal digits: short
  [invalid input] the key must be 8 characters (64 bits, one byte per
  character) or 16 hexadecimal digits; 5 character(s) given

Key - 8 characters or 16 hexadecimal digits: Cripto24
```

The two long messages are wrapped here to fit the page; on the terminal each is printed on one line.

## 5. Verification

Every value in section 4 agrees with the standard worked example of DES, and the tests pin all sixteen round keys, not only the first and the last that are usually quoted.

That check has a limit, though: it compares the program against one published example. A round key cannot be verified against anything on its own — it is 48 bits with no structure to inspect. The only independent evidence that the schedule is right is that a *complete* DES encryption built on these sixteen keys reproduces a known ciphertext, because a single wrong bit anywhere in the schedule changes the output completely. So `tests/des_reference.py` adds the data path of section 1.2 — `IP`, `E`, the eight S-boxes, `P` and $IP^{-1}$ — and drives it with the round keys this program produces:

| Key | Plaintext | Expected ciphertext |
|---|---|---|
| `133457799BBCDFF1` | `0123456789ABCDEF` | `85E813540F0AB405` |
| `0101010101010101` | `0000000000000000` | `8CA64DE9C1B123A7` |
| `0E329232EA6D0D73` | `8787878787878787` | `0000000000000000` |

All three are reproduced exactly. That file is a test fixture and is not part of task 2.5; it exists so that the claim "the sixteen round keys are correct" can be checked by running the tests rather than taken on trust.

The remaining tests cover properties that must hold for *any* K+, not only for the example: that PC-1 and PC-2 name distinct positions in range and discard exactly the eight documented ones, that the shifts add up to 28 and are single only in rounds 1, 2, 9 and 16, that every round key is 48 bits, that the sixteen are all different, that $C_{16} = C_0$, that flipping any parity bit of the key leaves all sixteen round keys unchanged, and that flipping a real key bit disturbs most of them.

```
$ python3 -m unittest discover -s tests
.........................
----------------------------------------------------------------------
Ran 25 tests in 0.002s

OK
```

## 6. Conclusions

The key schedule is the smallest part of DES and the one that explains the most about it. Three facts fall straight out of implementing it. The first is where the "64-bit key" of DES goes: PC-1 discards eight of those bits before anything else happens, and the test that flips each of them in turn and gets the same sixteen round keys shows directly that they carry nothing. The effective key is 56 bits, and that — not any weakness in the rounds — is what ended DES.

The second is that the schedule is built out of nothing but rotations and two fixed selections. There is no arithmetic in it at all, which is what makes the $C_{16} = C_0$ identity possible: the sixteen shifts were chosen to add up to 28 precisely so that the halves return to their starting position, which lets the same hardware generate the keys forwards for encryption and backwards for decryption. It is a design decision visible in a one-line check at the end of the program's output.

The third is how little separates the key schedule from a cipher that does nothing. Because the schedule is linear, a key whose K+ is all zeros produces sixteen identical all-zero round keys — the degenerate case the tests exercise — and the four DES weak keys arise in exactly that way, from halves that are constant under rotation. The strength of DES is not in the key schedule but in the S-boxes, and the key schedule's job is only to make sure every round sees a different 48 bits.

On the implementation side, the single most useful decision was to write `permute()` once, 1-based, in the same terms the standard uses, and let PC-1 and PC-2 — and later IP, E, P and $IP^{-1}$ in the verification fixture — all be calls to it. The tables are then the only thing that can be wrong, and deriving the discarded positions from the tables rather than writing them down turns a transcription error into a failing test. The verification was the other lesson: checking the program against a published worked example proves only that it matches that example, while running a full DES encryption on its output and reproducing three known ciphertexts is evidence that does not depend on having copied the reference values correctly in the first place.

## 7. Source code

The full source code is in the repository github.com/alo42/CS.

To run the program:

```
git clone git@github.com:alo42/CS.git
cd CS/LAB4
python3 main.py
```

To run the automated checks:

```
python3 -m unittest discover -s tests
```

The program needs only Python 3 and the standard library.